

Game DP Hint

This is [LeetCode 292 – Nim Game](#), and the key is to stop thinking about the individual moves and look for a **winning/losing pattern**.

Hint 1 – Start with small values

Write down who wins for the first few n :

n	Can Alice win?	Why?
1	✗	No valid x
2	✓	Alice chooses $1 \rightarrow 1$
3	✗	Alice must choose $1 \rightarrow 2$, then Bob wins
4	✓	Alice can choose $1 \rightarrow 3$
5	?	Try all possible divisors
6	?	Try all possible divisors
7	?	Try all possible divisors
8	?	Try all possible divisors

Don't code yet. Find the pattern first.

Hint 2 – Think in terms of DP/game states

For a number n , ask:

Is there at least one move that sends the opponent to a losing position?

So conceptually:

```
text
win(n) = true
if there exists a valid x such that win(n - x) == false
```

And if **every possible move** sends the opponent to a winning state:

```
text
win(n) = false
```

This is the standard **Game DP / Winning State vs Losing State** idea.

Hint 3 – Look specifically at divisors

For a given n , you can only subtract a divisor of n .

For example:

```
text
n = 6

valid x:
1, 2, 3

moves:
6 → 5
6 → 4
6 → 3
```

Now ask:

What are the winning/losing states among 3, 4, 5?

That's where the pattern starts becoming obvious.

Final hint – Don't overcomplicate it

Once you calculate enough small values, you'll notice that the answer depends on a **very simple mathematical property** of n .

Try to discover:

text

1 ✖
2 ✔
3 ✖
4 ?
5 ?
6 ?
7 ?
8 ?
9 ?
10 ?

Your goal: find the repeating pattern before looking for the formula.

If you want, send me your guessed pattern, and I'll tell you whether you're on the right track without giving away the answer.

The key observation is surprisingly simple: **Alice wins exactly when n is even.**

C++ Code

C++

```
class Solution {
public:
    bool divisorGame(int n) {
        return n % 2 == 0;
    }
};
```

Why does this work?

Let's understand the game instead of memorizing the code.

Case 1: n is even

Suppose:

text

$n = 6$

Alice can always choose:

text

$x = 1$

because 1 divides every number.

So:

text

$6 \rightarrow 5$

Now Bob gets an **odd** number.

From an odd number, every divisor is also odd.

For example:

text

$5 \rightarrow 4$ (subtract 1)
 $9 \rightarrow 8$ (subtract 1)
 $15 \rightarrow 14$ (subtract 1)

So Bob must subtract an **odd** number from an odd number:

text

$\text{odd} - \text{odd} = \text{even}$

Therefore Alice gets an even number again.

So Alice can keep maintaining this:

```
text
Alice gets even
  ↓
Alice subtracts an odd divisor
  ↓
Bob gets odd
  ↓
Bob subtracts odd divisor
  ↓
Alice gets even
```

Eventually Bob gets 1, where he has no valid move.

Therefore:

```
text
even n → Alice wins
```

Case 2: n is odd

Suppose:

```
text
n = 7
```

Every divisor of an odd number is odd.

So whatever Alice chooses:

```
text
7 - odd = even
```

That means Alice gives Bob an **even** number.

And from an even number, Bob can choose 1 and give Alice an odd number again.

So Bob can maintain the same strategy.

Eventually Alice reaches:

```
text
1
```

and loses.

Therefore:

```
text
odd n → Alice loses
```

The entire idea

```
text
n even → true
n odd  → false
```

Hence:

```
C++
return n % 2 == 0;
```

Complexity

Metric	Complexity
Time	$O(1)$
Space	$O(1)$

The important lesson here is **not the % 2 trick**. The important game-DP insight is recognizing that the positions can be classified into **winning states** and **losing states**, and the mathematical structure lets us reduce the entire game to parity.

Yes. In fact, this problem can absolutely be solved using **DP first**, and then you can derive the $n \% 2 == 0$ solution from the DP pattern.

The DP approach is actually useful for understanding *why* the mathematical solution works.

1. Define the DP state

Let:

```
C++
dp[i] = true
```

mean:

```
The player whose turn it is can force a win when the current number is  $i$ .
```

Otherwise:

```
C++  
dp[i] = false
```

means the current player will eventually lose if both play optimally.

2. Base case

When:

```
text  
n = 1
```

There is no valid x because:

```
text  
 $0 < x < 1$ 
```

is impossible.

So:

```
C++  
dp[1] = false;
```

3. DP transition

Suppose we're calculating $dp[n]$.

We try every valid divisor x :

```
text  
 $x < n$   
 $n \% x == 0$ 
```

After choosing x , the game becomes:

```
text  
 $n \rightarrow n - x$ 
```

Now it's the **opponent's turn**.

Therefore, if we can find **even one move** where the opponent reaches a losing state:

```
C++  
dp[n - x] == false
```

then the current player wins.

So:

```
C++  
dp[n] = true;
```

The transition is:

```
C++  
if (n % x == 0 && dp[n - x] == false)  
    dp[n] = true;
```

4. Bottom-up DP code

```
C++  
class Solution {  
public:  
    bool divisorGame(int n) {  
        vector<bool> dp(n + 1, false);  
  
        dp[1] = false;  
  
        for (int i = 2; i <= n; i++) {  
            for (int x = 1; x < i; x++) {
```

```

        if (i % x == 0 && dp[i - x] == false) {
            dp[i] = true;
            break;
        }
    }
}

return dp[n];
}
};

```

5. Let's dry run it

Take:

```

text
n = 5

```

We calculate:

dp[1]

No moves:

```

text
dp[1] = false

```

dp[2]

Possible move:

```

text
2 → 1

```

And:

```

text
dp[1] = false

```

Therefore:

```

text
dp[2] = true

```

dp[3]

Only valid divisor:

```

text
3 → 2

```

But:

```

text
dp[2] = true

```

So Alice cannot move Bob into a losing state.

```

text
dp[3] = false

```

dp[4]

Possible moves:

```

text
4 → 3    (x = 1)
4 → 2    (x = 2)

```

We already know:

```

text
dp[3] = false

```

Therefore Alice can choose:

```

text
4 → 3

```

So:

```
text
dp[4] = true
```

dp[5]

Only valid divisor:

```
text
5 → 4
```

But:

```
text
dp[4] = true
```

Therefore:

```
text
dp[5] = false
```

So we get:

```
text
n:    1  2  3  4  5  6  7  8
dp:   F  T  F  T  F  T  F  T
```

And suddenly the pattern appears:

```
text
odd  → false
even → true
```

6. Why DP reduces to the O(1) solution

This is the important part.

The DP gives us:

```
text
1 → F
2 → T
3 → F
4 → T
5 → F
6 → T
7 → F
8 → T
```

So:

```
text
dp[n] = (n % 2 == 0)
```

Therefore we don't actually need to build the entire DP array.

Instead of:

```
C++
vector<bool> dp(n + 1);
```

we can directly return:

```
C++
return n % 2 == 0;
```

DP vs optimized solution

Approach	Time	Space	Idea
Brute force simulation	Huge	—	Try game possibilities
DP	$O(n^2)$	$O(n)$	Calculate winning/losing states
Mathematical	$O(1)$	$O(1)$	Recognize even/odd pattern

The DP solution is better for learning because it teaches you the fundamental game-DP pattern:

A state is winning if there exists at least one move to a losing state.

Formally:

```
text
WIN(n) = exists move x such that WIN(n - x) = FALSE
```

And:

```
text
LOSE(n) = all possible moves lead to WIN states
```

That's the same pattern you'll see in many **Game DP / Minimax / Combinatorial Game** problems.

Yes. The **memoization (top-down DP)** version is a very natural way to write this problem.

The main difference from tabulation is:

- **Tabulation:** start from 1 and calculate `dp[2]`, `dp[3]`...`dp[n]`.
- **Memoization:** start from `n`, recursively explore the states we need, and store their answers.

Memoization Code

```
C++
class Solution {
public:
    bool solve(int n, vector<int>& dp) {
        // Base case
        if (n == 1)
            return false;

        // Already calculated
        if (dp[n] != -1)
            return dp[n];

        // Try every possible divisor
        for (int x = 1; x < n; x++) {
            if (n % x == 0) {
                // If opponent loses, current player wins
                if (solve(n - x, dp) == false) {
                    return dp[n] = true;
                }
            }
        }

        // Every possible move makes opponent win
        return dp[n] = false;
    }

    bool divisorGame(int n) {
        vector<int> dp(n + 1, -1);

        return solve(n, dp);
    }
};
```

How is this working?

Let's take:

```
text
n = 6
```

We call:

```
C++
solve(6)
```

We don't know whether 6 is winning, so we try its valid moves.

Move 1: `x = 1`

```
text
6 → 5
```

So we ask:

```
C++
solve(5)
```

Now calculate 5 .

The only divisor less than 5 is:

```
text
x = 1

5 → 4
```

So:

```
C++
solve(4)
```

For 4 , possible moves are:

```
text
4 → 3    x = 1
4 → 2    x = 2
```

Try 4 → 3 :

```
C++
solve(3)
```

For 3 :

```
text
3 → 2
```

So:

```
C++
solve(2)
```

For 2 :

```
text
2 → 1
```

And:

```
C++
solve(1) = false
```

because 1 has no moves.

Therefore:

```
text
solve(2) = true
```

because the player at 2 can move to losing state 1 .

Then:

```
text
solve(3) = false
```

because its only move goes to 2 , which is winning.

Then:

```
text
solve(4) = true
```

because:

```
text
4 → 3
```

and 3 is losing.

Then:

text

```
solve(5) = false
```

because its only move is:

text

```
5 → 4
```

and 4 is winning.

Finally:

text

```
6 → 5
```

and 5 is losing.

Therefore:

text

```
solve(6) = true
```

The recursion looks like this

For $n = 6$, conceptually:

text

```
solve(6)
├── 6 → 5
│   ├── solve(5)
│   │   ├── 5 → 4
│   │   │   ├── solve(4)
│   │   │   │   ├── 4 → 3
│   │   │   │   │   ├── solve(3)
│   │   │   │   │   │   ├── 3 → 2
│   │   │   │   │   │   │   ├── solve(2)
│   │   │   │   │   │   │   │   ├── 2 → 1
│   │   │   │   │   │   │   │   │   solve(1)
│   │   │   │   │   │   │   │   │   FALSE
```

Then the answers return upward:

text

```
solve(1) = FALSE
↓
solve(2) = TRUE
↓
solve(3) = FALSE
↓
solve(4) = TRUE
↓
solve(5) = FALSE
↓
solve(6) = TRUE
```

Where does memoization happen?

This line:

C++

```
if (dp[n] != -1)
    return dp[n];
```

Suppose we have already calculated:

text

```
solve(4) = true
```

and later another branch asks:

C++

```
solve(4)
```

We don't calculate it again.

We simply do:

C++

```
return dp[4];
```

That's the whole point of memoization.

Initially

text

```
dp = [-1, -1, -1, -1, -1, -1, -1]
```

After solving 6 :

text

```
dp = [-1, F, T, F, T, F, T]
```

So every state gets calculated only once.

The most important line

This is the heart of the entire problem:

C++

```
if (solve(n - x, dp) == false)
    return dp[n] = true;
```

Read it in English:

If I can make one move that puts my opponent in a losing state, then I am in a winning state.

And if we finish the loop without finding such a move:

C++

```
return dp[n] = false;
```

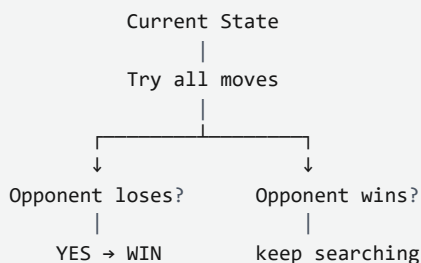
means:

Every move I make gives my opponent a winning position, so I lose.

The Game DP pattern to remember

For problems like this, memorize the logic, not the code:

text



So the general recurrence is:

text

```
dp[state] = TRUE
if there exists a move
such that dp[nextState] == FALSE
```

Otherwise:

text

```
dp[state] = FALSE
```

For this particular problem, after discovering the DP pattern, we can optimize everything down to:

C++

```
return n % 2 == 0;
```

But learning the memoization version first is the right way to understand why that one-line solution works.